

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- **EKS·GKE 프로덕션 비용 최적화** - 하이브리드 전환으로 20% 절감, GCP 마이그레이션(Fargate → GKE 노드 풀 재배포, MSP 리셀러 할인 확보)으로 50% 절감
- **그린필드 EKS 단독 구축·런칭** - 신규 외부 퍼블리싱 게임의 AWS 프로덕션 환경 - 플랫폼 컴포넌트 100% GitOps 선언적 관리, IRSA / Pod Identity로 정적 키 없는 인증, 2026.07 소프트 런칭 개시
- **IaC 표준화** - 온프레미스·AWS·GCP 전반을 Terraform·Ansible로 코드화 - 신규 컴포넌트 IaC 100%, 변경 이력 100% Git 추적, 동일 파이프라인 30분 내 재구축
- **AI 워크로드 GPU 인프라** - GCP A100 40GB 2장 모델 학습 서버를 Terraform IaC로 구축(너디스타), 온프레미스 GPU 서버에 사내 LLM 추론 서비스 운영(팬텀)

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- **파이프라인 설계·고도화** - GitHub Actions·GitLab CI·Jenkins·ArgoCD 기반 멀티클러스터 CI/CD 파이프라인 설계·배포 자동화
- **배포 생산성·신뢰성** - 배포 시간 67% 단축, GitOps 지속 배포 전환으로 시장 대응 속도 향상
- **카나리 무중단 롤아웃** - 프로덕션 게임 서비스를 Blue-Green → Argo Rollouts Canary로 전환해 승격 시 발생하던 503 레이스 제거
- **CI 튜체인 표준화** - 공용 튜체인 이미지 계층화와 자동 버전 사이클로 CI 빌드 환경 프로비저닝 5분 → 약 10초(약 97% 단축), 도구 버전 일관성 100%

모니터링 및 관측성 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- **선언적 로깅 인프라 현대화** - ELK → EFK → ECK Operator 기반 Elastic Stack 9.x CRD 선언적 관리 (TLS 자동 회전, 롤링 업그레이드 자동화)
- **메트릭 모니터링 체계** - kube-prometheus-stack 기반 커스텀 알림 규칙·Grafana 대시보드, 멀티 DB exporter(MySQL, Redis, Elasticsearch, PostgreSQL) 통합
- **분산 추적 비용·정확도 최적화** - OpenTelemetry + Tempo로 Metrics ↔ Traces ↔ Logs 3-signal 관측성 확보, 샘플링 지점 분리로 메트릭 정확도와 trace 저장 비용 절감

자동화 및 팀 역량 강화

Python·Bash 운영 자동화 도구 구축과, 고객사 운영팀이 스스로 운영하도록 만드는 교육·자료 표준화

- **운영 튜체인 자동화** - Shell → Python(표준 라이브러리 기반) 마이그레이션, wave 순차 자동 배포와 업그레이드 MR 자동 생성 파이프라인 구축
- **엔지니어링 교육·운영 정착** - DP사 운영팀 대상 Kubernetes 6개월 실습형 교육 설계·운영(수강자 평균 10명·회당 4시간), 자료 표준화로 타 고객사 재사용
- **오픈 엔지니어링 지속 실천** - 공개 API 10여 곳의 금융·경제 데이터를 수집·검증·재배포하는 파이프라인을 평일 하루 2회 자동 실행(**경제 대시보드**), 백엔드 없는 **다이어그램 에디터**도 공개 운영

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

네트워크·로깅 스택 전환, 공용 차트 오픈소스 공개, 업스트림 기여

- **Gateway API 마이그레이션** - Ingress-nginx 다수 인스턴스를 NGINX Gateway Fabric 단일 컨트롤플레인으로 마이그레이션 (사고 없이 전체 cutover, wildcard TLS 통합)
- **오픈소스 공용 자산 배포** - 오픈소스 Helm 차트 다수를 비롯해 Kubernetes CRD 컨트롤러, Composite GitHub Actions, Ansible 컬렉션 다수 공개 (ArtifactHub / GitHub Marketplace / Ansible Galaxy 등록)
- **CNCF 업스트림 기여** - apache/airflow · OpenTelemetry · prometheus-community 등의 Helm 차트에는 Gateway API HTTPRoute 지원 기능을, Go 프로젝트에는 버그 수정을 기여

인프라 보안 & IAM

Keycloak Operator SSO와 Vaultwarden으로 사내 인증·시크릿 거버넌스 일원화

- **중앙 인증(SSO/OIDC) 체계** - Keycloak Operator 도입으로 기존 GitLab 계정 기반 단일 로그인 구축, ArgoCD / Harbor / Vaultwarden OIDC 연동 및 검증 완료
- **사내 시크릿 거버넌스** - Vaultwarden을 Kubernetes에 Helm 배포, GitLab SSO·자동 백업·대화형 restore 스크립트로 중앙 관리·복구 체계 구축
- **GitOps 시크릿** - Git에 평문 커밋되던 DB 비밀번호를 External Secrets Operator + AWS Secrets Manager로 전환, Harbor pull secret은 Sealed Secrets로 표준화

주요 경력 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, dev-qa는 온프레미스에, review-prod는 AWS에 두는 하이브리드 아키텍처를 설계·구축·운영해 월 인프라 비용을 20% 절감했습니다.

라이브 게임은 외부 퍼블리셔와 협업해 2차 기술지원·운영을 담당하고, 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도: 하이브리드 아키텍처 설계·구축 100% (단독) / 라이브 게임 2차 기술지원 및 운영은 외부 퍼블리셔 운영팀과 협업 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS 단일 환경에서 운영하며 클라우드 비용 부담이 컸습니다. 트래픽 변동이 적은 비프로덕션 환경까지 클라우드 리소스를 상시 점유하고 있어 비용 최적화와 환경 분리가 필요했습니다.

접근 전략

- 하이브리드 아키텍처 설계** - 워크로드별 비용 및 사용 패턴을 분석해 dev-qa 환경은 온프레미스로 전환하고, 확장성과고가용성이 필요한 review-prod 환경은 AWS에 유지
- 네트워크 및ファイ프라인 격리** - 복잡한 사이트 간 VPN 구성 없이 애플리케이션 레벨에서 트래픽을 분리하고, 이미지 레지스트리로 이원화 (dev-qa=Harbor, review-prod=ECR)하여 환경별로 독립적으로 빌드·운영
- IaC 기반 운영 표준화** - 온프레미스와 AWS 전반에 Terraform과 Ansible을 도입해 신규 컴포넌트 IaC 100%를 달성하고, 이중 인프라 환경에서도 일관된 형상 관리 체계 수립
- IAM 자격증명 거버넌스** - 서비스·CI 계정만 코드로 관리하고 콘솔 계정은 분리. Terraform state 파일에 장기 자격증명(엑세스 키)이 평문으로 남지 않도록 사용자 키 자동 생성을 배제하고 개별 발급·회전 정책 적용

트리블슈팅

- 클라이언트 버전별 백엔드 분기 라우팅** - EKS 단일 Ingress 환경에서 구버전과 신버전 게임 클라이언트 요청을 서로 다른 백엔드로 분기해야 하는 요구사항 발생.
ALB HTTP 커스텀 헤더 기반의 조건부 라우팅 규칙과 타깃 그룹 매핑을 조합해 추가 인프라 구성 없이 유연한 분기 라우팅 구현
- 롤링 배포 시 L7 커넥션 단절 방지** - ALB 타깃 등록 해제 지연(deregistration delay)과 SIGTERM 타이밍이 어긋나 종료 중인 Pod로 신규 요청이 유입되거나 진행 중인 요청(in-flight)이 끊김. preStop 훅으로 헬스체크를 먼저 실패시켜 신규 유입을 차단하고 드레인 지연 동안 잔여 트래픽을 소진, RollingUpdate는 Pod 우선 생성(add-before-remove)으로 조정해 롤아웃 중 용량 유지 - 커넥션 단절 0건.
CloudWatch 기반 Grafana에서 ELB 5xx·Target 5xx를 분리해 배포 전후 대조, 5xx 0건·p95 유지 확인

성과

- 워크로드 분석 기반의 하이브리드 재배치로 월 인프라 비용 20% 절감 및 절감분의 신규 프로젝트 재투자에 기여
- 개발·QA와 프로덕션 환경을 물리적으로 분리해 장애의 영향 범위(blast radius)를 한쪽 환경 안에 가두고 운영 안정성 확보

프로젝트 2: 게임 애플리케이션 CI/CD 파이프라인 구축 및 공용 툴체인 표준화 (App GitOps)

기여도 100% (파이프라인·CI 표준화 전 과정 단독 설계·구축) 2024.12 ~ 현재 (운영 중)

게임 애플리케이션의 수동 빌드·배포를 GitLab CI와 ArgoCD 기반 GitOps 파이프라인으로 자동화하고, 그 위에 사내 repo 전체가 공유하는 CI 툴체인 이미지 계층과 버전 거버넌스를 표준화해 빌드 환경 프로비저닝과 도구 버전 관리를 한 흐름으로 묶었습니다.

2-1. 게임 애플리케이션 배포 CI/CD 파이프라인 (App GitOps, 2024.12 ~ 현재)

문제

개발자가 수동으로 빌드 후 서버에 직접 배포하는 방식이라 배포당 평균 30분이 소요되었고 휴먼 에러로 인한 서비스 장애가 잦았습니다.

접근 전략

- GitOps 기반 지속 배포(CD) 파이프라인 구축** - GitLab CI와 ArgoCD를 연계해 Git Push를 트리거로 빌드 → 테스트 → 배포 전 과정을 자동화하고, Git에 선언된 형상과 클러스터 상태 간 불일치를 감지해 자동 복구(Self-healing)하도록 구성. dev-qa·review-prod 4개 환경의 배포 로직을 단일 파이프라인으로 통합해 관리 일관성 확보
- 최소 권한 기반의 안전한 이미지 빌드 환경 수립** - Docker-in-Docker(DinD) 방식에서 요구되는 호스트 관리자 권한(Privileged 모드) 부여를 배제하기 위해, 도커 데몬 없이 사용자 영역(User-space)에서 컨테이너 이미지를 빌드하는 Kaniko를 도입해 CI 러너의 보안 취약점과 권한 탈취 리스크를 원천 차단

트리블슈팅

- ArgoCD 연쇄 삭제(cascading prune) 방지** - Git 커밋 오류로 하위 Application 전체가 연쇄 삭제되는 사고를 방지하기 위해, 워크로드 Application은 `prune: true` + `selfHeal: true` 로 자동 일치시키고 최상위 app-of-apps 루트·AppProject 계층은 `prune: false` 로 두어 리소스 삭제는

수동 검증을 거치도록 통제.

모든 sync 이벤트는 Slack 알림으로 실시간 추적

- **정적 데이터 배포 경로에서 SSH 키 제거** - CI 배포 계정이 SSH 키를 상시 보유한 채 대상 인스턴스에 직접 접속(scp/rsync)하던 구조를, 산출물을 S3 중계 버킷에 올리고 SSM SendCommand로 인스턴스가 스스로 내려받는 방식으로 교체해 배포 환경의 정적 SSH 키를 완전히 제거하고 자격증명 유출 리스크 차단

성과

- 배포 리드타임 67% 단축 (30분 → 10분) - 빌드·배포·설정 수동 단계를 파이프라인으로 일원화해 휴먼 에러로 인한 배포 장애 원천 제거
- 데이터 배포 소스 패치 시간 약 78% 단축 (약 139초 → 30초) - 최신 커밋과 필요한 파일만 선별해 받는 Git 최적화(shallow + blobless fetch)로 전송 병목 해소
- GitOps 기반 지속 배포 체계 정착 - 핵심 서비스 저장소별 주 100회 내외의 안정적인 배포 수행 및 신규 기능의 프로덕션 도달 시간 단축
- 다중 저장소 MasterData 파이프라인 구축 - 기획 데이터 변경 사항을 하위 서비스에 자동으로 일괄 전파하고, 파이프라인의 무한 반복 실행을 원천 차단

2-2. 공용 Toolchain Image 계층 구조 및 자동 버전 사이클 (2025.09 ~ 현재)

문제

개별 저장소(Repo)의 CI 잡마다 Alpine 베이스에 필수 도구(helm·helmfile·kubectl·crane·aws-cli)를 매번 설치해 빌드 환경 프로비저닝에 평균 5분이 소요되었습니다.

또한 도구 버전이 저장소별로 파편화되어 신규 버전 반영 시 전수 수정이 필요했고, `:latest` 태그 의존으로 인해 빌드 재현성과 보안 취약점 추적이 어려웠습니다.

접근 전략

- **Layer 1 (Mirror)**: 외부 레지스트리(Docker Hub, GHCR)의 도구 이미지를 Multi-arch 매니페스트를 유지한 채 `crane copy` 로 사내 Harbor에 미러링해 외부 의존성 제거
- **Layer 2 (Custom)**: Layer 1을 기반으로 사전 빌드된 공용 이미지(helmfile-tools, base-tools, node-build, rclone-backup, aws-cli-tools)를 구성해 개별 저장소가 이미지 태그 지정만으로 도구를 즉시 활용하도록 표준화
- **버전 관리 체계 (SSOT)**: 공용 빌드 템플릿(`.toolchain_build_custom`)과 중앙 버전 변수(`TOOLCHAIN_*_TAG`)를 정의해 하위 저장소가 개별 도구 태그를 직접 추적하지 않도록 일원화
- **공용 CI 템플릿 모듈화**: 개별 복사해 쓰던 CI 잡을 공용 템플릿 저장소로 통합하고 `include` 방식으로 참조.
공용 앵커(git-ssh-rules·job-config)와 주요 잡(정적 키 없는 AWS 인증, Kaniko 빌드, ArgoCD 배포, 백업)을 중앙에서 관리해 설정 드리프트(Configuration Drift) 원천 제거
- **자동 버전 사이클 구축**: Upstream 신규 버전 감지부터 하위 전파까지 자동화하되, 파이프라인 실행과 Tier 2 MR 승인 단계에 사람의 검증 게이트(Human-in-the-loop)를 배치해 안정성 확보:
 - (1) Upstream 신규 버전 감지 (주간 스케줄러)
 - (2) Layer 1 미러링 자동 MR 생성
 - (3) Layer 2 리빌드 및 중앙 버전 변수 동기화 자동 MR 생성
 - (4) 하위 저장소(Consumer Repos) 대상 일괄 자동 전파 (template provider repo 제외)

트러블슈팅

- **도구 간 버전 충돌로 인한 불체인 빌드 실패**: 자동 버전 상향 중 helmfile 1.5.1의 최소 요구 사양(helm 3.18.6 이상)과 기존 고정 버전(helm 3.16.4) 간의 불일치로 `build_custom` 빌드 파이프라인 장애 발생
- **원인 및 해결**: 도구 간 상호 의존성은 upstream 릴리스 노트 외에 사전 자동 감지가 불가능하다고 판단. helm·helmfile·kubectl에 minor-guard를 도입해 동일 minor 내 patch 버전만 자동 상향되도록 통제하고, minor·major 변경은 호환성 검증 후 수동 승격하도록 분리(`*_TARGET_MINOR` 환경 변수를 통한 1회성 계획적 오버라이드 지원)

성과

- **CI 프로비저닝 시간 약 97% 단축 (5분 → 약 10초)** - 사전 빌드된 표준 이미지 적용으로 빌드 지연을 해소하고 사내 저장소 전반의 도구 버전 일관성 100% 달성
- 체계적인 4-Tier 버전 거버넌스 수립 - 빌드 인자(ARG)부터 중앙 변수 관리, 컨슈머 분리, lint 기반 드리프트 감지까지 파편화된 버전 관리 지점을 단일 흐름으로 정돈
- 리스크 기반 순차 업그레이드 파이프라인 정착 - 컴포넌트별 리스크 티어(T1 주간 / T2 격주 / T3 월간 + pre-flight)에 맞춰 신버전 감지, Slack 알림, wave 순차 적용 자동화 구현(MAJOR_PIN 가드 및 T3 atomic 롤백 적용)
- 신버전 전파 자동화 대상은 helm·helmfile·kubectl·crane·rclone - 파급이 큰 이미지 리빌드는 스케줄이 아닌 사람 검토 게이트 뒤에 배치
- 멀티 아키텍처 환경 지원 - `docker buildx` 로 amd64·arm64 Multi-arch 이미지 빌드 체계 마련

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석까지 전 과정 단독 설계·구축) 2024.11 ~ 현재

로그가 분산된 온프레미스 환경에서 중앙 집중식 로깅 체계를 3단계로 고도화하고, 로그 수집 계층의 데이터 무결성과 고가용성(HA)을 강화한 뒤, 축적된 EFK 파이프라인을 게임 유저 리텐션 분석 플랫폼으로 확장했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

문제

온프레미스 환경에서 각 서버와 컨테이너 로그가 물리적으로 분산되어 있어 파드 재생성 시 휘발성 로그가 유실되었으며, 통합 관측 체계의 부재로 장애 발생 시 원인 분석과 복구(MTTR)에 많은 시간이 소요되었습니다.

접근 전략

- **수집·가공 아키텍처 경량화 (ELK → EFK)**: 노드별 Filebeat 수집 + 중앙 Logstash 가공 구조에서 JVM 기반 Logstash 티어의 과도한 메모리 부하를 해소하기 위해, 수집기는 경량 Fluent-bit DaemonSet으로, 파싱은 Fluentd로 역할을 분리해 전환
- **선언적 클러스터 관리 전환 (ECK Operator)**: 공식 Helm 차트의 업데이트 정체(2년 이상) 문제를 해결하고자 ECK Operator를 도입하고, CRD 기반의 선언적 라이프사이클 관리 체계 구축
- **클라우드(EKS) 확장 및 키리스(Keyless) 자동화**: 온프레미스 스택을 AWS EKS 환경에 맞춰 템플릿화해 배포하고, 인덱스 수명주기(ILM) 및 S3 스냅샷 백업 경로에 IRSA를 적용해 정적 자격증명 없이 자동화

트러블슈팅

- **ECK Admission Webhook 다운그레이드 거부로 인한 복구 장애**: 온프레미스 Elastic Stack 신규 버전 자동 갱신 중 릴리스 피드에는 등록되었으나 레지스트리에 미배포된 이미지를 참조해 `ImagePullBackOff` 발생.
즉각 롤백을 시도했으나 ECK Admission Webhook의 다운그레이드 거부 정책으로 복구가 차단되며 Operator 장애로 확산
- **원인 분석 및 개선**: 단순 배포 실패보다 '롤백 불능 상태'를 핵심 운영 리스크로 정의. 업그레이드 파이프라인에 이미지 레지스트리 실존 사전 검증, 클러스터 헬스 체크, 메이저 버전 상향 시 스냅샷 백업 경고, Webhook 제약을 고려한 안전 롤백 절차를 체계화해 재발 방지

성과

- 분산 노드 및 휘발성(Ephemeral) 파드 로그의 중앙 집계 체계 구축 - 파드 재생성·종료 시 발생하는 로그 유실을 원천 차단하고, Kibana 단일 뷰 기반의 시계열 로그 추적으로 장애 원인 규명 시간 단축
- 운영 자동화 및 Elastic Stack 메이저 버전 업그레이드 - TLS 인증서 갱신·계정 프로비저닝·노드 교체의 수동 작업을 자동화(CRD 선언 기반 롤링 업그레이드)하고, 2년 이상 정체되었던 Elastic Stack 메이저 버전 업그레이드 완료 (8.x → 9.x)

3-2. Fluent-bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 현재)

오프셋 영속화를 통한 데이터 무결성 확보: SQLite 오프셋 DB를 노드 로컬 스토리지(`hostPath`)에 저장해 수집기 파드가 재시작되어도 읽은 위치가 유지되도록 구성(재색인 및 중복 수집 0건 달성)

수집 정체(Stall) 감지 모니터링 구축: 프로세스는 다운되지 않고 로그 수집만 멈추는 비정상 정체(Stall) 상태를 감지하는 모니터링 및 알람을 연계해 미감지 로그 유실 방지

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

EFK 파이프라인의 제품 분석 플랫폼 확장: 장애 분석·검색용 로그 인프라를 확장해 Elasticsearch Transform(5분 주기)으로 인게임 이벤트를 사용자당 1행으로 집계

리텐션 지표 시각화: 별도 상용 SaaS 도입 없이 기존 EFK 파이프라인과 Kibana 대시보드만으로 D+1~D+30 리텐션 및 코호트 분석 환경 구축

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버 · 셀프서비스 운영 콘솔)

기여도 100% (전체 시스템 설계 및 개발) 2025.02 ~ 현재

개발팀의 반복적인 수동 작업을 자동화하고 생산성을 높이기 위해 6가지 사내 도구를 설계·개발·운영했고, 마지막에는 이들을 단일 인증 게이트 뒤의 셀프서비스 콘솔 플랫폼으로 묶었습니다.

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

Jenkins APK 빌드 완료 시 Slack으로 완료 메시지·다운로드 QR을 자동 전송하는 Python 봇을 Kubernetes에 배포 - QA 전달 시간 90% 단축, 버전 혼동 제거

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

GitLab push를 트리거로 문서를 Google Drive에 rclone으로 단방향 동기화 - `---checksum` 비교로 실제 변경분만 전송, 바뀐 디렉터리만 부분 clone(sparse)해 대용량 저장소 전송량 절감, 문서 관리 시간 80% 단축(10분 → 2분), GitLab이 단일 기준이고 Drive 측 수정은 다음 동기화 때 덮어쓰는 구조

4-3. 사내 LLM 서비스 구축 (4주, 2025.11)

온프레미스 GPU 서버에 Ollama·Open WebUI를 docker compose로 올려 사내 LLM 직접 운영(개발팀 사용)

4-4. Git 저장소 미러링 도구 개발 및 Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 현재)

CodeCommit·GitLab·GitHub 간 수동 동기화 누락과 지연을 해소하기 위해 Go 기반 미러링 도구 git-bridge 자체 개발 및 오픈소스화 - 저장소 간 동기화를 자동화하고, 실패 재시도 및 Slack 알람 연동으로 동기화 현황 실시간 추적. 이후 강제 덮어쓰기·삭제 등 데이터 유실 위험이 있는 작업 앞에 확인 가드를 두고

동기화 이력 콘솔을 구축해 무엇이 언제 어떻게 동기화됐는지를 사후 추적 가능하게 개선 - 이력 조회 범위를 기존 대비 10배(최대 5,000건) 확장해 장기 관측성을 확보하고, 프로세스 종료 시 진행 중인 동기화를 끝까지 기다리는 드레인(Graceful Shutdown, 실효 중앙값 2.9초)으로 배포 중 동기화 유실 차단. 사내에서는 소스용 신규 GitLab과 백업·외부 노출용 구 GitLab 두 인스턴스를 잇는 상시 동기화 경로로 운영 중

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

유지보수가 중단되고 핵심 기능(UI·검색·ZIP 일괄 다운로드)이 없던 기존 파일 서버를 대체하기 위해 Go 기반 정적 파일 서버 자체 개발 및 오픈소스 공개 - 자체 Helm 차트와 NFS 스토리지를 연계해 클러스터에 배포하고, APK 배포 파이프라인의 산출물 전달 경로로 기존 도구를 완전 대체

4-6. 사내 셀프서비스 운영 콘솔 플랫폼 구축 (2026.07 ~ 현재)

문제 정의

계정 생성·삭제 등 반복적인 운영 업무가 인프라 담당자만 실행할 수 있는 셀 스크립트에 의존해 모든 요청이 한 사람을 거치는 병목이 발생했습니다. 또한 10종의 사내 계정 시스템이 파편화되어 있어 특정 사용자의 잔여 계정과 권한 현황을 통합 추적·감사하기 어려운 구조적 한계가 있었습니다.

접근 전략

- **단일 OIDC 인증 게이트웨이 구축**: 개별 애플리케이션의 인증 구현 부담을 제거하고, 포털 전면엔 중앙 OIDC 게이트웨이를 배치해 인증된 트래픽만 백엔드로 인입되도록 설계(신규 서비스 추가 시 설정 등록만으로 즉시 연동)
- **사내 도구 및 운영 서비스 통합 포털화**: 기존 사내 웹 도구와 신규 운영 서비스 5종(계정·Keycloak·Harbor·Elasticsearch·VPN)을 포함한 총 7개 애플리케이션을 단일 진입점으로 통합 제공
- **계정 시스템 10종 연동 및 감사 로그 체계화**: 분산된 계정 시스템 10종의 프로비저닝 API를 콘솔에 연동해 통합 조회·제어를 지원하고, 모든 계정 변경 이력(생성·수정·삭제)의 작업 주체를 기록하는 감사 추적 체계 구축

트러블슈팅

- **CI 빌드와 GitOps 동기화 레이스 컨디션으로 인한 다운타임 발생**: 매니페스트 커밋 시점과 실제 컨테이너 이미지 빌드 완료 시점의 차이로 인해, ArgoCD가 레지스트리에 아직 존재하지 않는 이미지 태그를 동기화하여 `ImagePullBackOff` 발생. 특히 RWO 볼륨을 사용하는 서비스(전체 8개 중 6개)의 `Recreate` 배포 전략 특성상 기존 파드가 먼저 종료되며 전면 서비스 중단 초래
- **원인 분석 및 개선**: 동기화 실행 전 대상 이미지 태그를 실제로 pull해 보는 ArgoCD PreSync 훅을 구현. 이미지가 준비될 때까지 ArgoCD가 기존 Deployment를 건드리지 않도록 제어해 이미지 부재로 인한 전면 중단 제거

주요 성과

- **운영 병목 해소 및 셀프서비스 전환** - 6주간 단독으로 설계·개발해, 담당자 한 사람을 거쳐야 하던 계정·권한 운영 요청을 각 팀이 콘솔에서 직접 처리하도록 전환
- **중앙 거버넌스 뷰 및 단일 점점 통합** - 사내 핵심 도구 7개를 단일 OIDC 인증 게이트 뒤로 일원화하고, 계정 시스템 10종의 상태를 사용자 단위로 단일 화면에서 통합 조회·감사할 수 있는 환경 구축

프로젝트 5: Kubernetes 클러스터 · 사내 시스템 운영 고도화

기여도 100% (단독 설계 및 표준 절차를 자동화 스크립트로 코드화해 팀 내 재사용) 2025.12 ~ 현재

안정적인 클러스터 운영을 위해 모니터링과 Kubernetes 업그레이드·Helm 관리를 자동화하고, 네트워크 컨트롤러를 Gateway API로 전환했으며, 스토리지 병목 해소와 배포/업그레이드 파이프라인 정비까지 이어갔습니다.

이어서 클러스터 밖 사내 시스템 전반의 백업·복구 체계와, 오래 정체되어 있던 사내 GitLab 메이저 업그레이드까지 함께 맡았습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

kube-prometheus-stack 기반 커스텀 알림과 Grafana 대시보드를 설계하고 DB·Node Exporter·Cilium 메트릭을 통합했습니다. 알림 규칙을 컴포넌트 그룹 및 심각도(severity)별로 세분화하고, 상위 알림 발생 시 하위 알림을 억제(inhibit)해 동일 장애의 중복 통보를 차단했습니다.

이후 클러스터 외부에서 CRD가 삭제되어 Prometheus 자체가 중단되었음에도 인지하지 못했던 장애를 계기로, 감시자가 감시 대상과 같은 장애 도메인 안에 있으면 함께 실패한다는 구조적 한계를 도출하고, 클러스터 외부 Bastion의 cron watchdog이 5분 주기로 모니터링 스택의 생존을 능동 점검하도록 구성했습니다(클러스터 내부 CronJob 방식은 감시 대상과 운명을 공유하므로 기각).

5-2. Kubernetes 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

Kubespray 업그레이드 절차와 업그레이드 후 9개 필수 검증 항목(노드·etcd·API Server·인증서 만료 등)을 스크립트화하고, Helm 프레임워크 및 표준 템플릿 동기화로 버전 관리를 표준화해 검증 시간을 90% 단축하고 인증서 갱신 자동화로 만료 장애를 원천 제거했습니다.

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

온프레미스의 다중 ingress-nginx 인스턴스를 NGF 2.x 단일 컨트롤플레인과 클래스별 Gateway CR로 통합 전환했습니다.

MetalLB IP를 유지해 기존 DNS와 방화벽을 그대로 둔 채 사고 없이 cutover를 마쳤고(클래스당 30~60초), Wildcard 인증서 통합 적용으로 수동 갱신 공수를 50% 절감했습니다.

5-4. 스토리지 성능 최적화: etcd / DB SSD 마이그레이션 및 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

GitLab Runner(buildx)의 디스크 I/O 급증으로 HDD 기반 etcd 및 게임 DB에 초 단위 지연이 발생하는 병목을 진단하고, Control Plane과 DB 워크를 SSD로 이전함과 동시에 빌드 워크로드를 별도 물리 서버의 전용 VM으로 완전 격리해 게임 DB COMMIT 지연을 9~14초에서 ms 단위로 정상화하고, 빌드 중 인게임 접속 불가 및 서비스 지연 장애 0건을 달성했습니다.

5-5. Kubernetes 인프라 배포 자동화 및 멀티클러스터 GitOps(ArgoCD) 전환 (2026.03 ~ 현재)

문제 정의

Shell/awk 기반의 인프라 모노레포는 복잡한 YAML 조작과 환경별 분기 처리의 한계로 테스트 자동화가 불가능했고, CI 러너 중심의 Push 배포(`helmfile apply`)는 클러스터 상태 드리프트(Drift) 감지 불가 및 멀티클러스터 확장의 한계가 존재했습니다.

접근 전략

- **Python 표준 라이브러리 기반 툴링 리팩토링**: 서드파티 패키지 의존성을 배제한 Python3 모듈화 및 모듈별 1:1 단위 테스트(`unittest`) 체계를 도입해 로컬·CI 러너 간 실행 일관성 확보
- **단계별(Wave) 배포 및 업그레이드 자동화**: 컴포넌트 의존성에 따른 6단계 Wave 배포 체계를 수립하고, 차트 신버전 감지 시 Values diff 및 호환성 검증을 포함한 MR 자동 생성 파이프라인 구축
- **ArgoCD Pull-GitOps 컨트롤플레인 전환**: ApplicationSet(Matrix/Git Generator)으로 온프레미스·AWS 배포를 자동화하고, 기존 Helm 릴리스를 리소스 재생성 없이 ArgoCD 관리 상태로 무중단 이관(Adoption)
- **GitOps 거버넌스 및 안전장치 수립**: 최상위 계층 연쇄 삭제 방지(Cascading Prune 차단) 및 AppProject 허용 목록(Whitelist)을 적용해 장애 영향 범위(Blast Radius) 격리

주요 성과

- 온프레미스·AWS 멀티클러스터 플랫폼 릴리스를 단일 GitOps 컨트롤플레인에서 선언적으로 중앙 통제
- 레거시 Shell 스크립트를 단위 테스트가 보장된 Python 툴링으로 전면 마이그레이션해 배포 안정성 확보
- 차트 업그레이드 MR 자동화 및 Wave 배포를 통해 인프라 운영 오버헤드 대폭 감축
- 선언형 형상 자동 복구(Self-healing) 및 계층별 Prune 통제로 대규모 삭제 사고 위험 원천 차단

5-6. 백업·복구(DR) 체계 구축과 복구 검증 (2026.08 ~ 2026.09)

사내 시스템 8곳과 NAS 경로 2곳의 백업·복구 체계를 구축하고, 단순 '백업 수행'이 아닌 '실제 복구 가능 여부'를 기준으로 검증 프로세스를 정립했습니다. DB 덤프가 중간에 잘린 채로도 백업 작업이 정상 종료(Exit code 0)로 기록되어 실제 복원 시 실패하던 잠재 결함을 사전에 발견해 수정했고, 검증 기준을 '스크립트 실행 완료'에서 '복구된 데이터의 정확성 확인'으로 옮겨 재해 복구의 실질적 신뢰성을 확보했습니다.

5-7. 사내 GitLab 이원화 및 레거시 메이저 업그레이드: 21 hop 경로 (2024.10 신규 구축 / 2026.08 ~ 2026.09 업그레이드 6일)

입사 직후 신규 베어메탈 GitLab을 구축해 신규 프로젝트의 소스 저장소로 삼고, 필요한 기존 저장소는 GitLab mirroring으로 이관했습니다. 구 GitLab(컨테이너)은 백업·외부 노출 역할로 남기고, 두 인스턴스는 자체 개발한 git-bridge로 상시 동기화합니다.

이후 정체되어 있던 구 GitLab을 공식 업그레이드 경로를 따라 6일간 21회 hop으로 끌어올렸습니다. 버전을 건너뛰지 않고 경로를 그대로 밟은 것은, 중간 버전이 수행하는 마이그레이션을 건너뛰면 복구 경로가 사라지기 때문입니다. 첫 hop 다운타임 약 6분 30초(1회성 볼륨 이동 포함), 업그레이드 이후 야간 DR 복제본 123G → 39G(-68%).

프로젝트 6: 인프라 보안 및 거버넌스 체계 구축

기여도 100% (설계, 배포, SSO 연동) 2026.04 ~ 현재

파편화되어 있던 계정 인증·시크릿 관리·원격 접근 경로를 표준화하여 사내 보안 거버넌스를 일원화했습니다.

6-1. 중앙 인증(SSO/OIDC) 체계 일원화 (Keycloak Operator, 2026.04 ~ 현재)

사내 도구(Harbor·ArgoCD·Vaultwarden)마다 개별 연동되어 파편화되었던 GitLab OIDC 인증 경로를 Keycloak Operator 기반 중앙 IdP로 통합. GitLab Group → Keycloak Group Mapper → ArgoCD Role로 이어지는 권한 흐름을 일원화해 인증 정책 중앙 통제 및 단일 로그인(SSO) 환경 구축

6-2. 시크릿 거버넌스 수립: Vaultwarden · GitOps Secrets (2026.04 ~ 현재)

메일·메신저 등에 흩어져 있던 사내 공용 패스워드를 Kubernetes 기반 Vaultwarden으로 이관하고 SSO를 연동해 중앙 관리 체계 확립. 또한 Git에 평문 커밋되던 DB 접속 정보를 External Secrets Operator(AWS Secrets Manager 연동)로 전환하고, Harbor 클러스터 시크릿을 Sealed Secrets로 표준화하여 GitOps 환경의 자격증명 노출 취약점 원천 차단

6-3. 사내망 보안 원격 접속 환경 구축 (OpenVPN PKI, 2026.07)

동시 접속 50명 규모의 온프레미스 사내망 VPN 게이트웨이를 PKI 기반으로 구축하고, 사용자별 인증서 발급·폐기 전 과정을 먹통 셀 스크립트로 자동화

프로젝트 7: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (단독 설계·구축·운영) 2026.06 ~ 현재 (구축·운영 중)

신규 외부 퍼블리싱 게임의 상용 런칭을 위해 그린필드 EKS 프로덕션 환경을 단독 구축하고(7-1), 분산 추적(7-2)과 Canary 무중단 배포(7-3)를 보장해 소프트 런칭(7-4)까지 운영했으며, 실트래픽 위에서 SLI/SLO·에러 버짓(7-5)과 게임 CDN 엣지 보안(7-6)을 더했습니다.

7-1. 그린필드 EKS 프로덕션 인프라 구축 및 실트래픽 최적화 (2026.06 ~ 현재)

문제 정의

온프레미스 단일 클러스터 환경은 글로벌 트래픽 대응과 유연한 오토스케일링에 한계가 있어, 해외 상용 런칭을 위한 독립적인 고가용성 EKS 프로덕션 아키텍처 수립이 필요했습니다.

접근 전략

- **하이브리드 멀티클러스터 및 선언적 관리**: 그린필드 EKS 클러스터를 신설하고, 플랫폼 컴포넌트 전반을 GitOps(ArgoCD)로 선언적 일원화 관리
- **Karpenter 기반 컴퓨트 분리 최적화**: 게임 서비스는 On-Demand 노드로 안정성을 담보하고, CI 러너는 ARM 다중 패밀리 Spot 풀로 다변화해 비용 절감 및 인스턴스 회수율 최소화
- **네트워킹·인증 자동화**: AWS Load Balancer Controller, ExternalDNS, Wildcard ACM을 연계해 Ingress 및 TLS 인증서 발급 전 과정을 자동화하고, 전체 ServiceAccount에 Pod Identity/IRSA를 적용해 정적 IAM 키 제거
- **로깅·시크릿 기반 구축**: eck-operator 기반 HA EFK 스택(3노드 3-AZ, EBS gp3, S3 스냅샷 SLM)과 External Secrets Operator를 도입해 신규 클러스터의 로깅·시크릿 참조 경로 확보
- **배포 경로 자격증명 격리**: 클라이언트 아티팩트는 Jenkins → S3 → CloudFront 파이프라인으로 전송하고, 서버 매니페스트는 S3 중계 및 SSM SendCommand를 통해 Bastion EFS에 원격 전달(SSH 키 상시 보관 배제)

트리블슈팅

- **Karpenter 노드 축소와 게임 세션 보호 양립**: 노드 회수 완전 차단(`karpenter.sh/do-not-disrupt`)은 유틸 비용을 유발하므로 배제하고, 세션이 Redis로 외부화된 Stateless 구조를 활용해 preStop drain + PDB(`minAvailable: 1`) + Disruption Budget(`nodes: 1`) 조합으로 1노드씩 안전한 순차 축소 달성
- **롤링 배포 후 유틸 노드 미회수 개선**: 메모리 Request 과다 설정(실측 101MiB 대비 2GiB)으로 발생한 노드 간 점유 불균형(92% vs 35%) 및 자동 축소 실패를 진단. Request를 512MiB로 현실화하고 Consolidation 조건을 조정해 빈패킹(Bin-packing) 밀도와 자동 노드 회수 정상화
- **신규 Pod 기동 중 트래픽 유입 차단**: Pod 기동 중 ALB가 트래픽을 선제 전달해 발생하던 간헐적 5xx 에러를 ALB Pod Readiness Gate 도입으로 해결. Target Group 헬스체크 통과 후에만 Pod를 Ready로 판정하도록 제어

주요 성과

- 해외 런칭을 위한 독립 AWS 하이브리드 멀티클러스터 구축 및 플랫폼 매니페스트 100% GitOps 선언적 관리 확립
- Bastion SSH 키를 비롯한 인프라 전반의 정적 자격증명 완전 제거
- 프로덕션과 심사용 워크로드를 단일 NodePool로 통합해 풀 분리 시 풀마다 기본 점유하던 최소 1노드의 비용 낭비 제거 및 bin-packing 밀도 극대화
- GitOps 매니페스트 및 CI/CD 전환 절차를 표준 런북으로 문서화하여 팀 내 인수인계 및 동일 환경 재구축이 가능하도록 함

7-2. OpenTelemetry 엔드투엔드 분산 추적 파이프라인 (OTel + Tempo, 2026.06 ~ 현재)

문제 정의

기존 관측 체계(EFK 로깅, Prometheus 메트릭)만으로는 마이크로서비스 간 호출 흐름과 지연 시간(Latency) 병목 구간, 분산 환경의 에러 전파 경로를 파악하는 데 한계가 있었습니다.

접근 전략

- **추적 스택 및 무침습 계측 환경 구축**: AWS EKS에 OpenTelemetry Collector와 Grafana Tempo(S3 백엔드, EKS Pod Identity로 정적 키 없이 연동)를 구축하고, OTel Operator Auto-instrumentation으로 애플리케이션 코드 수정 없이 언어별 SDK 주입
- **샘플링 지점 분리를 통한 관측 정확도·스토리지 비용 최적화**: RED 메트릭은 샘플링 전 100% Span에서 추출해 집계 정확도를 보장하고, 저장은 Tail Sampling(에러 전체 + 500ms 초과 요청 + 정상 요청 10%)으로 스토리지 비용 절감

주요 성과

- **3-Signal 상호 연계**: Metrics·Traces·Logs를 Grafana 단일 뷰에서 오갈 수 있게 연결해 서비스 간 호출 흐름과 병목 구간을 가시화(기존 로깅은 프로젝트 3, 메트릭은 5-1)
- **2계층 라우팅으로 트레이스 파편화 해결**: Collector 다중 replica에서 앱 Pod마다 gRPC 커넥션이 서로 다른 인스턴스에 고정되어, Pod 경계를 넘는 트레이스의 Span이 흩어지면 '에러 트레이스 전량 보존'과 Service Graph 조인이 깨질 수 있는 구조적 결함 식별
- 수신 계층(Layer 1)과 샘플링 계층(Layer 2)으로 분리하고 Headless Service + Trace ID 기반 라우팅(loadbalancing exporter)을 적용해, 한 트레이스의 Span이 항상 같은 샘플링 인스턴스에 모이도록 보장
- 실트래픽 검증: 커넥션 고정으로 replica별 시리즈 수가 1,650 : 541로 치우쳐 있던 분담이 Trace ID 해싱 후 초당 Span 4.09 : 5.18로 고르게 분산
- **샘플링 계층 알림 공백 보강**: 2계층 분리 후 기존 Collector 다운 알림이 샘플링 계층의 전면 장애를 감지하지 못하는 공백을 확인하고, 샘플링 계층 다운·버퍼 초과로 판정 전 버려지는 트레이스·버퍼 임계 근접·정책 평가 오류 알림 4종을 신설해 조용한 트레이스 유실을 감시

7-3. Argo Rollouts 기반 Canary 무중단 배포 전환 (Blue-Green → Canary, 2026.06 ~ 현재)

문제 정의

기존 Blue-Green 배포 방식은 승격(Promotion) 시 ALB Target Group을 교체하는 과정에서 일시적으로 Healthy Target이 0이 되는 공백이 생겨 간헐적 503 에러가 발생하는 한계가 있었습니다.

접근 전략

- **Canary 롤아웃 전환**: 동일 ALB Target Group 내에서 `maxUnavailable: 0` 및 add-before-remove(신규 Pod 기동 완료 후 구버전 회수) 전략을 적용해 구/신버전 일시 공존을 통한 503 레이스 컨디션 제거
- **Stateless 구조 전제**: 게임 서비스가 stateless HTTP(JWT + Redis/TypeORM으로 상태 외부화)라 전환 중 old/new에 트래픽이 섞여도 안전함을 확인. 다만 old/new가 동시에 떠 있으면 안 되는 변경(API 스펙 변경 등)은 이 전략 밖이라 정기점검 시간에 처리
- **배포 진입 및 종료 라이프사이클 동기화**: ALB Pod Readiness Gate를 통해 트래픽 수신 준비가 끝난 Pod만 트래픽 풀에 편입하고, 종료 시 Deregistration Delay(30초) → preStop(35초) → TerminationGracePeriod(60초) 순차 드레인 시퀀스를 확립해 In-flight 요청의 무결성 보장 (SIGTERM에서 스스로 in-flight 요청을 정리하는 .NET 서비스만 75초)
- **배포 전략 모듈화 및 컨트롤러 HA**: 공용 Helm 차트 내 배포 전략을 템플릿화해 프로덕션 게임 메인 서비스 17개에 Canary를 적용(공개 트래픽이 없는 같은 게임의 내부 서비스는 일반 Deployment 유지)하고, 컨트롤러 Leader Election HA 및 PDB를 구성해 노드 재배포 중에도 안정적인 롤아웃 지원

주요 성과

- 프로덕션 게임 배포 시 발생하던 ALB Healthy Target 0 공백 및 503 레이스를 원천 차단, 하위 호환 패치는 무중단 배포

7-4. 소프트 런칭 운영 (2026.07 ~ 현재)

성과

- 2026.07.14(KST) 소프트 런칭 개시 — 제한 공개로 연 소규모 실트래픽을 앞서 구축한 Canary 배포·3-signal 관측 위에서 안정적으로 받아냄
- 2026 Q4 글로벌 정식 런칭에 필요한 관측·배포·오토스케일링(HPA·Karpenter)을 모두 갖추(소프트 런칭 규모에서는 아직 스케일아웃이 걸리지 않음)
- 소프트 런칭 2주간의 실트래픽 관측 데이터(피크 445 RPS, OOMkill 0건)를 기반으로 Pod requests·limits 재산정 - 과도한 request로 인해 HPA 스케일아웃이 트리거되지 않던 구조를 정상화하고, CPU 48%(2,200m → 1,150m)·메모리 71%(6GiB → 1.75GiB) 절감

7-5. SLI/SLO·에러 버짓 정의 (2026.08)

문제 정의

소프트 런칭 후 실트래픽이 유입되었으나, '정상 서비스 상태'에 대한 정량적 합의 기준이 부재했습니다. 이로 인해 장애 판단이 주관적 직관에 의존했고, 시스템 안정화 투자와 개선의 우선순위를 결정할 객관적 근거가 부족했습니다.

접근 전략

- **실측 가능한 신호 기반의 SLI 수립**: 이론적인 지표를 먼저 정해 데이터를 끼워 맞추는 방식을 지양하고, 현재 텔레메트리로 실제 연산이 성립하는지 사전 검증한 유효 신호에 한해 SLI를 기록 - SLO 목표와 에러 버짓은 대시보드에서 선택해 비교하도록 두고, 확정 목표치는 기준선이 쌓인 뒤 정하도록 의도적으로 유보
- **왜곡된 가용성 SLI의 의도적 배제**: 게임 서버가 예외 발생 시에도 HTTP 200을 반환하여, 수집된 약 790만 건의 Span 중 에러가 0건으로 잡히는 계측 결함 확인.
이 상태에서 가용성 지표를 만들면 장애 상황에서도 '가짜 100%(False Green)'를 띄워 에러 버짓이 소진되지 않으므로, 애플리케이션 계측이 정상화되기 전까지 도입을 유예

주요 성과

- 실트래픽 검증을 통과한 지연시간 신호만 SLI로 기록하고, 왜곡된 가용성 지표는 대시보드에서 배제해 장애를 정상으로 오인할 위험을 원천 차단
- 가용성 SLI 도입을 가로막던 애플리케이션의 예외 처리 방식(에러 시 HTTP 200 반환)을 정량적 데이터로 특정하고, 지표 정상화를 위한 선행 수정 요건을 명확히 제시

7-6. 게임 CDN 엣지 보안: AWS WAFv2 적용 (2026.07)

접근 전략

- **Count 모드 선행 검증을 통한 오탐 방지**: AWS 관리형 룰 그룹(AMR) 4종을 Count 모드로 배포해 실트래픽에서 어떤 요청이 매칭되는지 규칙 단위로 관측하고, 오탐 확인 → 세부 규칙 예외 → 차단(Block) 전환으로 이어지는 절차를 tfvars 한 줄 플립으로 실행되도록 설계 - 검증 없는 차단으로 발생할 수 있는 정상 유저가 막히는 일(False Positive) 및 CS 인입 리스크 원천 차단
- **WCU 용량 및 비용 예측 가능성 확보**: 기본 상한(1,500 WCU) 이내인 1,125 WCU로 규칙 용량을 통제하고, 고정 비용을 월 약 \$9 수준으로 사전 산정하여 예산 범위 내에서 안정적 도입
- **Rate-based Rule의 의도적 도입 유예**: 모바일 통신사 CGNAT 환경 특성상 다수 유저가 동일 IP로 집약되어 오탐 위험이 높고, 당시 실측 피크가 5분당 28 reqs 수준에 불과해 즉시 도입을 보류 - 무리한 차단 대신 트래픽 규모에 따른 단계적 재검토 기준을 문서화

주요 성과

- 게임 CDN 앞단에 WAFv2를 정상 유저 영향 없이 도입 - 관리형 룰 4종을 실트래픽 Count 모드로 관측 중이며, 차단 전환 조건과 미도입 룰(Rate-based)의 활성화 기준을 기술적 근거와 함께 문서로 정리

프로젝트 8: EKS Auto Mode 2차 클러스터 및 Site-to-Site VPN(WireGuard) 게이트웨이 구축

기여도 100% (단독 설계·구축) 2026.08 ~ 현재 (구축·전환 중)

문제 정의

신규 퍼블리싱 게임이 퍼블리셔 내부 인증 계정과 통신해야 했고, 이 구간은 공용 인터넷을 경유할 수 없어 VPC 피어링을 통한 사설 경로가 요구되었습니다. 그런데 기존 프로덕션 VPC의 대역이 피어링 조건과 맞지 않아 VPC와 그 위의 클러스터·데이터 계층까지 전면 재생성이 불가피했고, 재구축되는 클러스터는 퍼블리셔의 요청에 따라 EKS Auto Mode로 구성해야 했습니다.

기존 클러스터는 관리형 노드그룹 위에 Karpenter를 직접 운영하는 구조라 컴퓨트 영역의 Terraform·state 관리 포인트가 비대했고, 노드 수명주기·스팟 중단·핵심 애드온을 직접 관리하는 오버헤드도 컸습니다. 또한 VPC 피어링은 중계 라우팅을 지원하지 않아(non-transitive) 온프레미스에서 AWS 내부 ALB(private)로 가는 사설 경로는 별도로 확보해야 했습니다.

접근 전략

- EKS Auto Mode 기반 컴퓨트 계층 관리형 전환:** 노드 프로비저닝, 수명주기 관리, Karpenter 운영에 필요한 AWS 리소스(IAM·SQS·EventBridge) 및 내장 애드온(LB Contoller·EBS CSI)을 AWS 완전 관리형으로 이관하고 선언적 NodePool만 유지하여 인프라 코드 복잡도 대폭 감감
- IPAM 제약 대응 플랫폼 재설계(Re-platforming):** 가용 서브넷 확장 불가 제약(/20 4개가 상위 /18을 정확히 소진)에 맞춰, 서브넷 물리 분리 대신 단일 private 서브넷 내 보안 그룹(Security Group) 기반 논리 격리 아키텍처로 전환(Aurora·Valkey·EFS 재배치)
- Site-to-Site WireGuard VPN 게이트웨이 구축:** 퍼블릭 ALB를 하나 더 두면 같은 서비스가 호스트명·인증서·DNS 레코드를 따로 갖게 되고 환경이 하나 늘 때마다 그 세 가지가 함께 늘어나므로, 대신 터널링을 택해 단일 도메인·인증서 체계를 유지한 채 모든 내부 트래픽이 사설 경로로 통신하도록 설계. 온프레미스에서 AWS(EIP) 방향 아웃바운드로 터널을 수립하고, Peer Endpoint를 비워 첫 핸드셰이크 과정에서 상대 주소를 동적으로 학습하도록 구성
- 플랫폼 컴포넌트 호환 마이그레이션:** 관측성·보안·CI 툴체인 등 16개 핵심 플랫폼 컴포넌트를 Auto Mode 사양에 맞게 재구성하고, 10개 컴포넌트를 ArgoCD Pull-GitOps 관리 체계로 전환

트러블슈팅

- Auto Mode 시스템 테인트로 인한 부트스트랩 순환 교착 해소:** 기본 system NodePool의 `CriticalAddonsOnly` 테인트 때문에 자체 워크로드 노드가 프로비저닝되기 전까지 일반 Pod 용량이 0이라 ArgoCD Pod부터 Pending으로 멈추는 데드락 발생. NodePool·IngressClass 매니페스트를 GitOps 트랙에서 분리해 helmfile 기반 사전 부트스트랩 단계에서 선제 배포하도록 배포 순서 재정립
- IngressClass 누락에 따른 로드밸런서 미생성(silent failure) 해결:** AWS Load Balancer Controller Helm 차트가 프로비저닝하던 IngressClass가, 컨트롤러를 내장한 Auto Mode에는 그 차트가 없어 만들어지지 않음. `ingressClassName: alb` 를 명시해도 매칭되는 컨트롤러가 없어 에러 로그도 없이 로드밸런서가 생성되지 않는 현상을 식별하고, Auto Mode 전용 IngressClass를 독립 컴포넌트로 정의해 클러스터 생성 직후 선행 배포
- NodeClass API 그룹 변경 및 미지원 애드온 분리:** Auto Mode에서도 NodePool 작성은 그대로지만 NodeClass의 API 그룹이 `karpenter.k8s.aws/v1` EC2NodeClass에서 `eks.amazonaws.com/v1` NodeClass로 바뀌어 기존 매니페스트가 호환되지 않음. 스펙을 신규 CRD에 맞춰 재작성하고, Auto Mode가 내장하지 않는 EFS CSI Driver는 배포를 별도로 분리

주요 성과

- 컴퓨트 계층 Terraform 관리 포인트를 102개 → 17개로 83% 감축하고, 노드 수명주기·스팟 회수 오버헤드를 AWS 완전 관리형으로 이관
- 온프레미스·AWS 간 Site-to-Site VPN 게이트웨이를 구축해 내부 ALB의 공용 인터넷 노출 없이, 환경이 늘어나도 단일 내부 호스트명과 단일 TLS 인증서 체계를 그대로 유지
- 정적 라우트를 Pod·노드 단위가 아닌 게이트웨이 라우터 한 지점으로 모아, 경로가 사라지면 일부가 아니라 전체가 곧바로 실패하는 즉시 실패(fail-fast) 방식으로 구성해 Pod 배치에 따라 서비스 가용성이 엇갈리는 부분 장애(gray failure)를 제거
- `terraform plan` 으로 감지되지 않는 터널의 외부 의존 요소 세 가지(라우터 정적 경로·게이트웨이 설정 파일·개인키)를 런북에 문서화해 운영 가시성을 확보하고, 개인키는 tfstate 버킷에 평문으로 남지 않도록 AWS SSM Parameter Store SecureString으로 분리

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임·블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며 AWS에서 GCP로 대규모 클라우드 마이그레이션을 총괄했습니다.

비용 최적화와 글로벌 네트워크 성능 확보를 목표로, 게임 점검 시간을 활용해 최소 다운타임으로 전체 서비스를 이전했습니다. 소스 저장소는 온프레미스는 GitLab, 프로덕션은 GitHub로 이원화해 관리했고, 정체돼 있던 베어메탈 GitLab을 공식 경로를 따라 다단계(약 12 hop)로 제자리 업그레이드했습니다. 또한 MongoDB·CloudSQL·GA·Dune 등 멀티 소스 데이터를 BigQuery로 모아 Google Sheets까지 자동 갱신하는 데이터 수집 파이프라인을 구축해, 분석가의 일 2~3시간 수동 작업을 없앴습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 아키텍처 설계·마이그레이션 전략 수립 및 실행 총괄 70% (나머지 30%는 서버 개발팀의 애플리케이션 측 마이그레이션 협업), CI/CD 파이프라인 구축 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용이 지속 상승했고, 특히 상시 가동되는 게임 워크로드를 Fargate로 돌리며 발생하는 Pod 단위 과금이 큰 비중이었습니다. 비용 절감과 멀티 리전 게임 서비스를 위한 글로벌 로드밸런싱 확보를 목표로 GCP 마이그레이션을 추진했습니다.

접근 전략

- 3단계 순차 이전** - 1단계 개발 환경 GCP 구축·검증, 2단계 QA, 3단계 프로덕션

- **네트워크·인증** - Shared VPC(Host Project / Service Project 분리) 설계, GitHub Actions의 GCP 인증은 Workload Identity Federation으로 전환해 서비스 계정 키 발급·순환을 OIDC 단계 토큰으로 대체.
DNS는 서브도메인 위임(Hosting.kr → AWS Route53 → GCP Cloud DNS)으로 사전 검증하고, 최종 NS 변경만 점검 공지 후 cutover
- **컴퓨터 재배치** - AWS Fargate 워크로드를 GKE 표준 노드 풀로 이전. 상시 워크로드는 Pod 단위 과금이 VM 단위보다 불리해 동일 과금 구조인 GKE Autopilot 제외
- **스토리지 제약 우회·WAF** - Filestore SSD 최소 2.5TB 제약은 Compute Engine에 pd-balanced 1TB 디스크를 붙인 NFS 서버 직접 구축으로 우회, Cloud Armor로 리전 차단·IP 화이트리스트 WAF(웹 방화벽) 정책 구성
- **laC·GitOps 파이프라인** - Terraform으로 GCP 인프라 laC화(IAM 제외 100%), 이전 후 GitLab CI + ArgoCD GitOps CI/CD 파이프라인 구축, Helm Chart로 환경별 설정 표준화

트러블슈팅

- AWS ALB 기반 라우팅을 GCP 글로벌 로드밸런서로 전환하는 과정에서, 백엔드가 모두 unhealthy일 때 AWS ALB는 fail-open으로 트래픽을 통과시키지만 GCP LB는 이를 차단하는 차이 때문에 트래픽 라우팅 문제 발생.
GCP NEG(Network Endpoint Group) 구조를 분석하여 GKE 워크로드에 맞는 헬스체크·백엔드 설정으로 재구성
- GKE Ingress + BackendConfig의 healthcheck requestPath 응답이 누락되어 503 에러 발생. 해당 경로 응답을 보장하고 ManagedCertificate / FrontendConfig 설정과 서로 맞춰 외부 진입 트래픽 정상화
- Cloud CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드 실패.
URL Map을 HTTP / HTTPS 두 개로 분리(default_url_redirect.https_redirect=true)하고 HTTP forwarding rule에 80 포트 target HTTP proxy를 별도 매핑하여 해결
- Cloud DNS 위임 후 Google Search Console에 도메인이 자동 등록되지 않아 검색 노출·소유권 확인 문제 발생. Search Console verification TXT 레코드를 Cloud DNS에 추가해 소유권 검증 후 재등록

성과

- 클라우드 비용 50% 절감 - AWS → GCP 마이그레이션 설계·수행, Fargate → GKE 표준 노드 풀 전환, MSP 리셀러 할인 확보
- 3단계 전략으로 전체 서비스 이전 완료 - 리소스 복사 방식으로 다운타임 최소화, 최종 DNS 전환은 2시간 내외 점검 공지 후 실행
- 전체 GCP 인프라를 Terraform으로 laC화하고, 배포 이력을 Git 커밋으로 100% 추적 가능하게 함
- Workload Identity Federation 도입으로 GitHub Actions의 GCP 서비스 계정 키 관리 부담과 키 유출 리스크 제거

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% (Python 스크립트 개발 및 운영) 3개월 (2024.01 ~ 2024.03)

문제

게임·블록체인 데이터를 수동으로 수집하다 보니 분석가가 매일 2~3시간을 소비했고, 휴먼 에러로 인한 데이터 누락도 빈번했습니다.

접근 전략

- **멀티 소스 BigQuery 적재** - MongoDB는 Dataflow Flex Template ETL, CloudSQL은 BigQuery 직접 connection, Google Analytics · Dune은 각자 API(Dune은 별도 API Key)로 적재, 일관된 인덱스 스키마로 정규화
- **분석가용 Google Sheets 자동 갱신** - Python Cloud Function이 BigQuery 쿼리 결과를 Google Sheets API로 적재, Cloud Scheduler가 Daily(전일)·Monthly(전월) 두 주기로 자동 트리거
- **전체 인프라 Terraform laC** - BigQuery Dataset · Cloud Function · Scheduler · Cloud Storage · Artifact Registry · IAM · Service Account를 코드로 관리

트러블슈팅

- CloudSQL → BigQuery 직접 connection(federated query)이 리전 불일치와 connection Service Account 권한 부족으로 실패. connection을 동일 리전에 생성하고 connection SA에 Cloud SQL Client 권한을 부여하여 해결
- Dune API의 execute → poll → fetch 비동기 실행 구조와 요청 한도로 결과 누락·타임아웃 발생. 실행 상태 폴링과 백오프로 결과 수신을 보장하고 API Key를 Secret Manager로 분리
- Daily 재실행 시 동일 레코드가 BigQuery에 중복 적재되는 문제 발생. 중복 제거 Cloud Function에 MERGE 및 ROW_NUMBER() 기반 정합성 로직을 적용해 적재를 멍등화(재실행해도 결과 동일)하고 데이터 누락·중복 0건 유지
- BigQuery 결과를 Google Sheets API로 적재할 때 단일 시트 1천만 셀 한도·쓰기 쿼터와 분당 요청 한도(429)에 걸려 갱신 실패. values.batchUpdate 청크 분할과 시트 분할로 셀 한도를 우회하고, 호출 pacing(throttle)·배치 처리·지수 백오프로 쿼터 내에서 동작하도록 구현

성과

- 데이터 수집 자동화 95% 달성 (일 2~3시간 수동 작업 제거), 휴먼 에러로 인한 데이터 누락률 0%
- 전체 인프라 Terraform laC화로 동일 파이프라인을 다른 GCP 프로젝트에 30분 내 재구축 가능, 변경 이력은 Git으로 100% 추적

프로젝트 3: AI 워크로드용 GPU 인프라 표준화 및 확장 (Windows → Linux · 세팅 자동화 · 사내 / GCP / 외부 IDC)

기여도 100% (인프라 설계·구축·운영) 6개월 (2024.02 ~ 2024.07)

문제

사내 GPU 서버가 Windows로 운영돼 이미지 생성 모델(Stable Diffusion 계열)을 돌릴 때마다 엔지니어가 실행 환경을 직접 세팅해야 했습니다. 드라이버·CUDA·프레임워크 조합을 손으로 맞추다 보니 같은 환경을 재현하기 어려웠고, 장비는 있는데 쓰기까지 시간이 걸렸습니다.

접근 전략

- **Windows → Linux 표준화** - 사내 GPU 서버를 Ubuntu 22.04로 전환, 세팅 스크립트가 드라이버 스택(nvidia-driver-535 · CUDA 11.5 · cuDNN 8.6)과 nvidia-docker 설치·검증까지 처리해 동일 환경 재현.
이미지 생성 워크로드(Stable Diffusion WebUI · LoRA 학습용 kohya_ss · ComfyUI)를 표준 구성으로 배치
- **GCP 클라우드 확장** - NVIDIA A100 40GB 2장 VM(a2-highgpu-2g)을 Terraform IaC로 구축, 리전·존별 A100 가용성 확인과 GPU 쿼터 상향을 거쳐 앞선 마이그레이션의 Shared VPC 서버넷에 연결해 네트워크 표준 유지
- **외부 IDC 20대 증설·워크로드별 카드 분리** - 같은 세팅 스크립트로 GPU 서버 20대 구축.
학습은 모델·배치가 카드 하나에 들어가지 않아 VRAM과 NVLink가 필요하고 추론은 요청마다 독립적이라 대수에 비례해 처리량이 늘기 때문에, 학습·대용량 작업은 GCP A100, 대량 추론은 IDC의 소비자용 RTX 40 시리즈로 분리 배치
- **퍼블리셔 협업 고도화** - 신규 퍼블리싱 계약 체결로 수요가 이어져 퇴사 시점까지 퍼블리셔 측과 협업하며 GPU 환경 고도화

성과

- Windows 수동 세팅을 Linux + 자동화 스크립트로 대체해, 1대용 절차가 외부 IDC 20대를 포함한 GPU 플릿 전체의 표준 구축 절차가 됨
- GPU 서버를 손으로 구축하지 않고 Terraform 코드로 관리해 재구축·변경 추적을 표준화 - 기존 GCP IaC 저장소에 그대로 편입
- GPU 자원을 사내 1대에서 사내 10대 · GCP A100 · 외부 IDC 20대로 확장하고, 워크로드(다중 GPU 학습 vs 요청 단위 추론)에 맞춰 카드 등급 분리 배치

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했습니다. 금융권·공공기관 프라이빗 클라우드 인프라를 설계·구축하고, 고객사 운영팀 교육을 직접 맡았습니다.

프로젝트 1: 고객사 맞춤형 PaaS 솔루션 구축

기여도 100% (인프라 설계 및 Kubernetes 클러스터 구축) 11개월 (2022.04 ~ 2023.03)

문제

고객사마다 요구하는 환경과 서버 스펙이 상이하여, 표준화된 솔루션으로는 대응이 어려웠습니다.

접근 전략

- **HA 클러스터 설계** - 고객사 요구사항에 맞춰 Kubernetes 클러스터를 HA(High Availability) 구성으로 설계하고 OpenStack으로 가상머신 관리 환경 구축
- **통합 스토리지** - Ceph로 블록·파일·오브젝트 스토리지 통합 제공

트러블슈팅

- **인증서 만료 주기 개선** - Kubernetes 클러스터 인증서의 1년 주기 만료로 인한 서비스 장애 리스크 및 매년 반복되는 갱신 작업(고객사당 연간 약 5시간) 발생. kubeadm 내장 갱신 명령(certs renew)의 특정 버전(v1.15~1.16) 미갱신 버그를 분석하여, 버전에 영향받지 않는 10년 유효기간의 자체 인증서 재생성 및 컴포넌트 재기동 스크립트(update-kube-cert)를 수립해 전 고객사에 장애 없이 적용

성과

- 5개 고객사에 HA 구성 PaaS 솔루션 구축·납품 후 안정 운영
- Ansible 기반 서버 프로비저닝·구성 관리 자동화로 고객사별 배포 시간 단축, 환경 불일치(Configuration Drift) 0건 달성
- 인증서 만료 주기 1년 → 10년 연장으로 만료 장애 차단, 고객사 10곳 기준 연간 약 50시간 절감

프로젝트 2: DP사 Kubernetes 운영 교육 프로그램 설계·운영

기여도 100% (교육 설계 및 진행) 6개월 (2022.06 ~ 2022.11)

문제

DP사 운영팀이 Kubernetes·컨테이너 기반 인프라 운영 경험이 부족하여, PaaS 솔루션 도입 후 자체 운영에 어려움이 예상되었습니다.

접근 전략

- **단계별 커리큘럼** - Kubernetes 기초부터 클러스터 운영·트러블슈팅까지 단계별 교육 커리큘럼을 설계하고 실습 환경을 구성해 실무 중심으로 운영
- **PaaS 스택 병행 교육** - OpenStack·Ceph 스토리지 운영 교육을 병행해 전체 PaaS 스택 운영 역량 확보

성과

- 수강자 평균 10명 · 회당 4시간 커리큘럼으로 DP사 운영팀의 Kubernetes 자체 운영 체계를 확립하고, 교육 후 고객사 기술 문의 감소
- 교육 자료를 표준화하여 이후 타 고객사 교육에도 재사용